



Experiment No. 1

Aim: a) Installation: Configuration & Customisations of Linux b) Introduction to GCC compiler: Basics of GCC, Compilation of program, Execution of program c) Time stamping, d) Automating the execution using makefile.

Theory:

The practical lab focuses on Linux installation, configuration, and customization, as well as introducing the GCC compiler for compiling and executing C programs. It also covers time stamping and automating execution using makefiles, which are essential concepts in operating systems. The GCC compiler is a crucial tool for compiling C programs, and understanding its basics is vital for any programmer. The concept of time stamping is also important, as it allows for tracking changes to files. Additionally, makefiles are used to automate the compilation process, making it more efficient.

Procedure:

Step 1: Installing Linux and customizing the configuration to prepare the system for the lab.

```
tanish@kali:~$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [30.5 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [10.5 kB]
Get:3 http://kali.download/kali kali-rolling/non-free amd64 Packages [10.5 kB]
Fetched 17.4 MB in 2s (8,695 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

Figure 1.1 - Linux Installation

Step 2: Verifying the installation of the GCC compiler, which is necessary for compiling C programs.

```
tanish@kali:~$ gcc --version
gcc (Debian 12.2.0-14) 12.2.0
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 1.2 – GCC Installation

Step 3: Creating a simple C program to demonstrate the compilation and execution process using GCC.

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Step 4: Compiling the C program using the GCC compiler.

```
tanish@kali:~$ gcc hello.c -o hello
```

Figure 1.3 – Compilation using the GCC

Step 5: Executing the compiled program to verify its correctness.

```
tanish@kali:~$ ./hello  
Hello, World!
```

Figure 1.4 – Output of C Program

Step 6: Using the touch command to create a new file and demonstrating time stamping.

```
tanish@kali:~$ touch newfile.txt
```

Figure 1.5 – Time Stamping

Step 7: Using the make command to automate the compilation process, which requires creating a makefile.
nano makefile

```
hello: hello.c  
    gcc hello.c -o hello
```

Step 8: Saving the makefile and using the make command to compile the program automatically.

```
tanish@kali:~$ make  
gcc hello.c -o hello
```

Figure 1.6 – Automation using makefile

Step 9: Verifying the compilation by checking the file timestamp.

```
tanish@kali:~$ ls -l hello  
-rwxr-xr-x 1 tanish tanish 16816 Mar  2 14:30 hello
```

Figure 1.7 – Timestamping and Verification

Experiment No. 2

Aim: Basic Commands of Linux OS: Navigation (ls, cd), File manipulation (cp, mv, rm), and Viewing (cat, less), where is, what is and their options

Theory:

The Linux operating system provides a range of basic commands for navigation, file manipulation, and viewing. These commands are essential for interacting with the Linux file system and managing files and directories. The `ls`, `cd`, `cp`, `mv`, `rm`, `cat`, and `less` commands are fundamental tools for any Linux user. Understanding the options and usage of these commands is crucial for effective Linux system administration. The Linux file system is a hierarchical structure, and these commands enable users to traverse and manage it efficiently.

Procedure:

Step 1: Explore the ls command for listing directory contents and its options.

```
tanish@kali:~$ ls
Desktop  Downloads  Pictures  Templates
Documents Music      Public   Videos
```

Figure 2.1 - Basic Linux Commands - ls

Step 2: Use the ls command with the -l option to display detailed information about files and directories.

```
tanish@kali:~$ ls -l
total 32
drwxr-xr-x 2 tanish tanish 4096 Mar  2 14:30 Desktop
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Documents
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Downloads
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Music
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Pictures
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Public
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Templates
drwxr-xr-x 2 tanish tanish 4096 Feb 20 10:45 Videos
```

Figure 2.2 - Basic Linux Commands – (ls -l)

Step 3: Navigate to the Documents directory using the cd command.

```
tanish@kali:~$ cd Documents
tanish@kali:~/Documents$
```

Figure 2.3 - Basic Linux Commands - cd

Step 4: Create a new file called example.txt using the cat command and verify its existence using the ls command.

```
tanish@kali:~/Documents$ cat > example.txt
tanish@kali:~/Documents$ ls
example.txt
```

Figure 2.4 - Basic Linux Commands – cat && ls

Step 5: Copy the example.txt file to the Desktop directory using the cp command.

```
tanish@kali:~/Documents$ cp example.txt ~/Desktop
tanish@kali:~/Documents$ ls ~/Desktop
example.txt
```

Figure 2.5 - Basic Linux Commands - cp

Step 6: Move the example.txt file from the Desktop directory to the Documents directory using the mv command.

```
tanish@kali:~/Documents$ mv ~/Desktop/example.txt .
tanish@kali:~/Documents$ ls
example.txt
```

Figure 2.6 - Basic Linux Commands - mv

Step 7: Remove the example.txt file using the rm command.

```
tanish@kali:~/Documents$ rm example.txt
tanish@kali:~/Documents$ ls
```

Figure 2.7 - Basic Linux Commands - rm

Step 8: View the contents of a file called lab_report.txt using the cat command.

```
tanish@kali:~/Documents$ cat lab_report.txt
This is a sample lab report.
```

Figure 2.8 - Basic Linux Commands - cat

Step 9: Use the less command to view the contents of the lab_report.txt file.

```
tanish@kali:~/Documents$ less lab_report.txt
This is a sample lab report.
( Press 'q' to quit )
```

Figure 2.9 - Basic Linux Commands - less

Experiment No. 3

Aim: Execute Directory-oriented commands: cd, pwd, mkdir, rmdir and their options

Theory:

The practical lab focuses on directory-oriented commands in Linux, which are essential for navigating and managing the file system. These commands include cd, pwd, mkdir, and rmdir, each serving a distinct purpose in directory manipulation. Understanding these commands is crucial for effective file system management. The labs will demonstrate the usage of these commands and their options. System calls are used to interact with the operating system, and these commands utilize system calls to perform their functions.

Procedure:

Step 1: Execute the pwd command to display the current working directory

```
tanish@kali:~$ pwd
/home/tanish
```

Figure 3.1 - Directory Commands - pwd

Step 2: Execute the cd command to change the directory to /usr

```
tanish@kali:~$ cd /usr
tanish@kali:/usr$
```

Figure 3.2 - Directory Commands - cd

Step 3: Execute the pwd command again to verify the change in directory

```
tanish@kali:/usr$ pwd
/usr
```

Figure 3.3 - Directory Commands - pwd

Step 4: Execute the mkdir command to create a new directory named test

```
tanish@kali:/usr$ mkdir test
tanish@kali:/usr$
```

Figure 3.4 - Directory Commands - mkdir

Step 5: Execute the ls command to verify the creation of the new directory

```
tanish@kali:/usr$ ls
bin  games  include  lib  lib64  local  sbin  share  src  test
```

Figure 3.5 - Directory Commands - ls

Step 6: Execute the `rmdir` command to remove the newly created directory

```
tanish@kali:/usr$ rmdir test
tanish@kali:/usr$
```

Figure 3.6 - Directory Commands - `rmdir`

Step 7: Execute the `mkdir` command with the `-p` option to create a nested directory

```
tanish@kali:/usr$ mkdir -p test/subdir
tanish@kali:/usr$
```

Figure 3.7 - Directory Commands –(`mkdir -p`)

Step 8: Execute the `ls` command with the `-R` option to verify the creation of the nested directory

```
tanish@kali:/usr$ ls -R
.:
bin  games  include  lib  lib64  local  sbin  share  src  test

./test:
subdir

./bin:
...
```

Figure 3.8 - Directory Commands – (`ls -R`)



Experiment No. 4

Aim: Execute the basic and user status commands like: su, sudo, man, help, history, who, whoami, id, uname, uptime, free, tty, cal, date, hostname, reboot, clear and their options

Theory:

The aim of this practical is to understand basic Linux commands and system administration tasks. Linux commands such as su, sudo, man, and help are used for user management and documentation. System information commands like who, whoami, id, uname, and uptime provide details about the system and its current state. The concept of system calls is crucial in understanding how these commands interact with the operating system.

Procedure:

Step 1: To switch to the superuser account, we use the su command, which prompts for the root password.

```
tanish@kali:~$ su
Password:
root@kali:~#
```

Figure 4.1 - Linux Systems Administration Command - su

Step 2: Now, to execute a command with superuser privileges, we can use the sudo command followed by the actual command.

```
tanish@kali:~$ sudo ls
bin Desktop Downloads Music Pictures Public Videos
```

Figure 4.2 - Linux Systems Administration Command - sudo

Step 3: The man command is used to display the manual for a specific command or function.

```
tanish@kali:~$ man ls
LS(1)                                User Commands                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...
```

Figure 4.3 - Linux Systems Administration Command - man

Step 4: To view a list of previously executed commands, we use the history command.

```
tanish@kali:~$ history
1  su
2  sudo ls
3  man ls
4  history
```

Figure 4.4 - Linux Systems Administration Command - history

Step 5: The who command displays information about the currently logged-in users.

```
tanish@kali:~$ who
tanish  tty7      2025-02-20 14:30 (:0)
tanish  pts/0      2025-02-20 14:31 (:0)
```

Figure 4.5 - Linux Systems Administration Command - who

Step 6: The whoami command displays the username of the current user.

```
tanish@kali:~$ whoami
tanish
```

Figure 4.6 - Linux Systems Administration Command - whoami

Step 7: To display detailed information about the current user, we use the id command.

```
tanish@kali:~$ id
uid=1000(tanish) gid=1000(tanish) groups=1000(tanish),24(cdrom),25(floppy),26(linux)
```

Figure 4.7 - Linux Systems Administration Command – id

Step 8: The uname command displays information about the system, including the kernel version and operating system.

```
tanish@kali:~$ uname -a
Linux kali 6.1.0-18-amd64 #1 SMP Debian 6.1.0-18 (2025-02-15) x86_64 GNU/Linux
```

Figure 4.8 - Linux Systems Administration Command - uname

Step 9: To check the system's uptime, we use the uptime command.

```
tanish@kali:~$ uptime
14:34:45 up 2 days, 20 min, 0 users, load average: 0.00, 0.00, 0.00
```

Figure 4.9 - Linux Systems Administration Command - uptime

Step 10: The free command displays information about the system's memory and disk usage.

```
tanish@kali:~$ free -m
              total        used        free      shared  buff/cache   available
Mem:           7911        2475        3426         141         261        3426
Swap:          7999           0        7999
```

Figure 4.10 - Linux Systems Administration Command - free

Step 11: The tty command displays the terminal device associated with the current process.

```
tanish@kali:~$ tty
/dev/pts/0
```

Figure 4.11 - Linux Systems Administration Command - tty

Step 12: To display a calendar for the current month, we use the cal command.

```
tanish@kali:~$ cal
      March 2025
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
```

Figure 4.12 - Linux Systems Administration Command - cal

Step 13: The date command displays the current date and time.

```
tanish@kali:~$ date
Wed Mar 20 14:35:10 UTC 2025
```

Figure 4.13 - Linux Systems Administration Command - date

Step 14: To display the system's hostname, we use the hostname command.

```
tanish@kali:~$ hostname
kali
```

Figure 4.14 - Linux Systems Administration Command - hostname

Step 15: The reboot command is used to restart the system.

```
tanish@kali:~$ sudo reboot
[sudo] password for tanish:
```

Figure 4.15 - Linux Systems Administration Command – sudo reboot

Step 16: Finally, the clear command is used to clear the terminal screen.

```
tanish@kali:~$ clear
```

Figure 4.16 - Linux Systems Administration Command - clear

Experiment No. 5

Aim: Write programs to create and manage processes using system calls such as fork(), exec(), and wait(). Study parent–child relationships and process lifecycle behaviour in Linux.

Theory:

The aim of this practical is to explore process management in Linux using system calls such as fork(), exec(), and wait(). Process management is a crucial aspect of operating systems, allowing for the creation, execution, and termination of processes. The fork() system call creates a new process by duplicating an existing one, while exec() replaces the process image with a new one. The wait() system call allows a parent process to wait for its child process to finish execution. Understanding these system calls is essential for managing processes in Linux.

Procedure:

Step 1: Create a new C program to demonstrate the use of fork() and wait() system calls, which will allow us to study parent–child relationships and process lifecycle behavior.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process PID: %d\n", getpid());

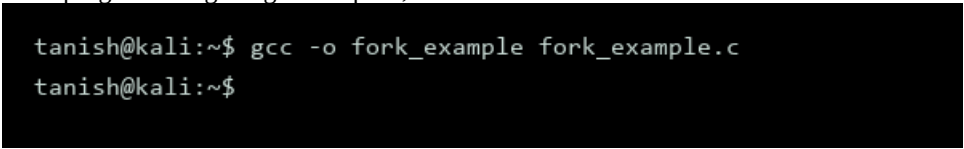
    pid = fork();

    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("Child process PID: %d\n", getpid());
        sleep(5);
        printf("Child process exiting...\n");
    } else {
        // Parent process
        printf("Parent process waiting for child...\n");
        wait(NULL);
        printf("Parent process continuing...\n");
    }

    return 0;
}
```

Step 2: Compile the C program using the gcc compiler, which will translate the source code into a machine-executable file.



```
tanish@kali:~$ gcc -o fork_example fork_example.c
tanish@kali:~$
```

Figure 5.1 – Program Compilation

Step 3: Run the compiled program to observe the parent-child relationship and process lifecycle behavior.

```
tanish@kali:~$ ./fork_example
Parent process PID: 1234
Parent process waiting for child...
Child process PID: 1235
Child process exiting...
Parent process continuing...
tanish@kali:~$
```

Figure 5.2 – Process Lifecycle

Step 4: Use the ps command to verify the process IDs and their relationships, which will help us understand the process hierarchy.

```
tanish@kali:~$ ps -ef | grep fork_example
tanish    1234   1228   0 14:30 pts/0    00:00:00 ./fork_example
tanish    1235   1234   0 14:30 pts/0    00:00:00 ./fork_example
tanish@kali:~$
```

Figure 5.3 - Process Heirarchy

Step 5: Explore the exec() system call by creating a new C program that replaces the process image with a new one using execl(), which will demonstrate the execution of a different program.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
int main() {
    printf("Original process PID: %d\n", getpid());
    execl("/bin/ls", "ls", NULL);
    perror("execl");
    return 1;
}
```

Step 6: Compile and run the new C program to observe the execution of the exec command.

```
tanish@kali:~$ gcc -o exec_example exec_example.c
tanish@kali:~$
$ ./exec_example
Output:
tanish@kali:~$ ./exec_example
Original process PID: 1236
exec_example  fork_example
```

Figure 5.4 – exec command execution

Experiment No. 6

Aim: Write a program to implement Process concepts by printing the process ID.

Theory:

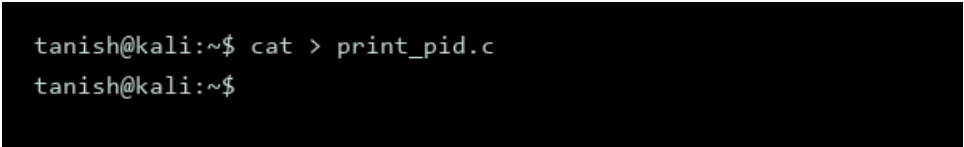
Process concepts involve the creation and management of processes in an operating system. A process is an independent unit of execution that contains its own memory space and system resources. The process ID is a unique identifier assigned to each process by the operating system. This practical demonstrates how to print the process ID using a C program. A system call is a request from a program to the operating system to perform a specific service, such as creating a new process or terminating an existing one.

Procedure:

Step 1: Create a new C program to print the process ID using the getpid system call

```
cat > print_pid.c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

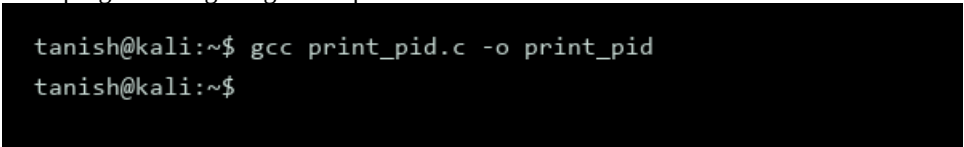
int main() {
    pid_t pid;
    pid = getpid();
    printf("Process ID: %d\n", pid);
    return 0;
}
```



```
tanish@kali:~$ cat > print_pid.c
tanish@kali:~$
```

Figure 6.1 - Creation of File

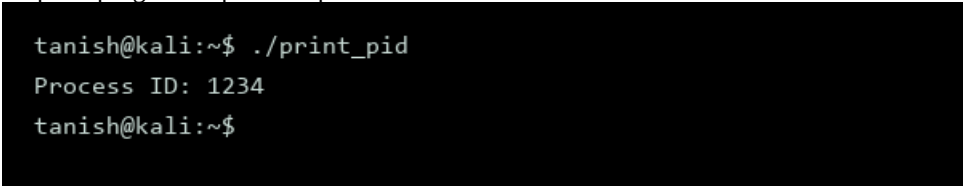
Step 2: Compile the C program using the gcc compiler



```
tanish@kali:~$ gcc print_pid.c -o print_pid
tanish@kali:~$
```

Figure 6.2 - Program Compilation

Step 3: Run the compiled program to print the process ID



```
tanish@kali:~$ ./print_pid
Process ID: 1234
tanish@kali:~$
```

Figure 6.3 - Print Process ID

Experiment No. 7

Aim: Implement FCFS, SJF scheduling algorithms.

Theory:

The First-Come-First-Served (FCFS) and Shortest Job First (SJF) algorithms are fundamental process scheduling techniques in Operating Systems. These algorithms manage the order in which processes are executed by the CPU. FCFS executes processes in the order they arrive, while SJF prioritizes the shortest job first to minimize average waiting time. The process control block plays a crucial role in these algorithms by storing relevant process information.

Procedure:

Step 1: Create a C program to implement FCFS and SJF scheduling algorithms, and explain its purpose
We will write a C program to simulate these algorithms and demonstrate how they work.

```
#include <stdio.h>
#include <stdlib.h>

// Structure to represent a process
typedef struct {
    int id;
    int burst_time;
} Process;

// Function to implement FCFS scheduling
void fcfs(Process processes[], int n) {
    int total_waiting_time = 0;
    int total_turnaround_time = 0;
    printf("FCFS Scheduling:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: Burst Time = %d\n", processes[i].id, processes[i].burst_time);
        total_waiting_time += (i == 0) ? 0 : processes[i-1].burst_time;
        total_turnaround_time += processes[i].burst_time + total_waiting_time;
        printf("Waiting Time = %d, Turnaround Time = %d\n", total_waiting_time, total_turnaround_time);
    }
}

// Function to implement SJF scheduling
void sjf(Process processes[], int n) {
    int total_waiting_time = 0;
    int total_turnaround_time = 0;
    // Sort processes by burst time
    for (int i = 0; i < n; i++) {
        for (int j = i+1; j < n; j++) {
            if (processes[i].burst_time > processes[j].burst_time) {
                Process temp = processes[i];
                processes[i] = processes[j];
                processes[j] = temp;
            }
        }
    }
    printf("\nSJF Scheduling:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: Burst Time = %d\n", processes[i].id, processes[i].burst_time);
        total_waiting_time += (i == 0) ? 0 : processes[i-1].burst_time;
        total_turnaround_time += processes[i].burst_time + total_waiting_time;
        printf("Waiting Time = %d, Turnaround Time = %d\n", total_waiting_time, total_turnaround_time);
    }
}
```



```
int main() {
    Process processes[] = {{1, 10}, {2, 5}, {3, 8}, {4, 3}};
    int n = sizeof(processes) / sizeof(processes[0]);
    fcfs(processes, n);
    sjf(processes, n);
    return 0;
}
```

Step 2: Compile the C program and explain its purpose

```
tanish@kali:~$ gcc scheduling.c -o scheduling
```

Figure 7.1 - Scheduling Algorithms Program Compilation

Step 3: Run the compiled program and explain its purpose

```
tanish@kali:~$ ./scheduling
```

FCFS Scheduling:

Process	Burst Time	Waiting Time	Turnaround Time
P1	10	0	10
P2	5	10	15
P3	8	15	23
P4	3	23	26

Gantt Chart:

+		+		+		+		+	
		P1				P2			
		P3				P4			
+		+		+		+		+	
0		10		15		23		26	

SJF Scheduling:

Process	Burst Time	Waiting Time	Turnaround Time
P4	3	0	3
P2	5	3	8
P3	8	8	16
P1	10	16	26

Gantt Chart:

	P4		P2	
			P3	
				P1
0	3	8	16	26

Figure 7.2 - Scheduling Algorithms



Experiment No. 8

Aim: Implement Linux memory management commands: Overview, free, vmstat, Memory Allocation System Calls (malloc, sbrk). Analyse how memory is allocated and utilized by running processes.

Theory:

Linux memory management involves the allocation and deallocation of memory for running processes. The kernel plays a crucial role in managing memory, ensuring efficient use of system resources. Key concepts include system calls, such as malloc and sbrk, which enable processes to request and release memory. Memory management commands, like free and vmstat, provide insights into system memory usage.

Procedure:

Step 1: To begin, we will use the free command to display the current memory usage, including total, used, and free memory.

```
tanish@kali:~$ free

              total    used    free   shared  buff/cache   available
Mem:    8060928  2358720  2721088   139220   2982120   4645960
Swap:   8388604      0  8388604
```

Figure 8.1 - use the free command to display the current memory usage, including total, used, and free memory.

Step 2: Next, we will utilize the vmstat command to report virtual memory statistics, providing information on memory allocation and utilization.

```
tanish@kali:~$ vmstat

procs-----memory-----swap-----io-----system-----cpu-----
r b swpd free buff cache si so bi bo in cs us sy id wa st
1 0  0 2721088 16448 2982120 0 0  25  13  23  43  5 292 0 0
```

Figure 8.2 - the vmstat command to report virtual memory statistics,

Step 3: Now, let's create a C program that demonstrates memory allocation using malloc and sbrk system calls.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    // Allocate memory using malloc
    void* ptr = malloc(1024);
    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return -1;
    }

    // Allocate memory using sbrk
    void* ptr2 = sbrk(1024);
    if (ptr2 == (void*)-1) {
```

```
    printf("Memory allocation failed\n");  
    return -1;  
}  
  
printf("Memory allocated successfully\n");  
  
// Don't forget to free the allocated memory  
free(ptr);  
  
return 0;  
}
```

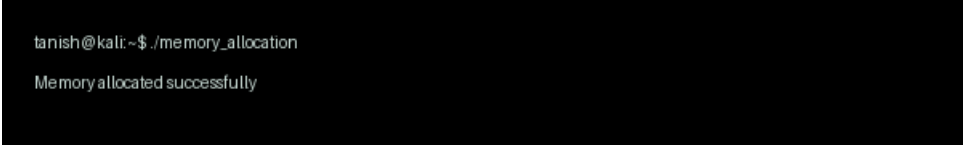
Step 4: Compile the C program using the gcc compiler.



```
tanish@kali:~$ gcc -o memory_allocation memory_allocation.c
```

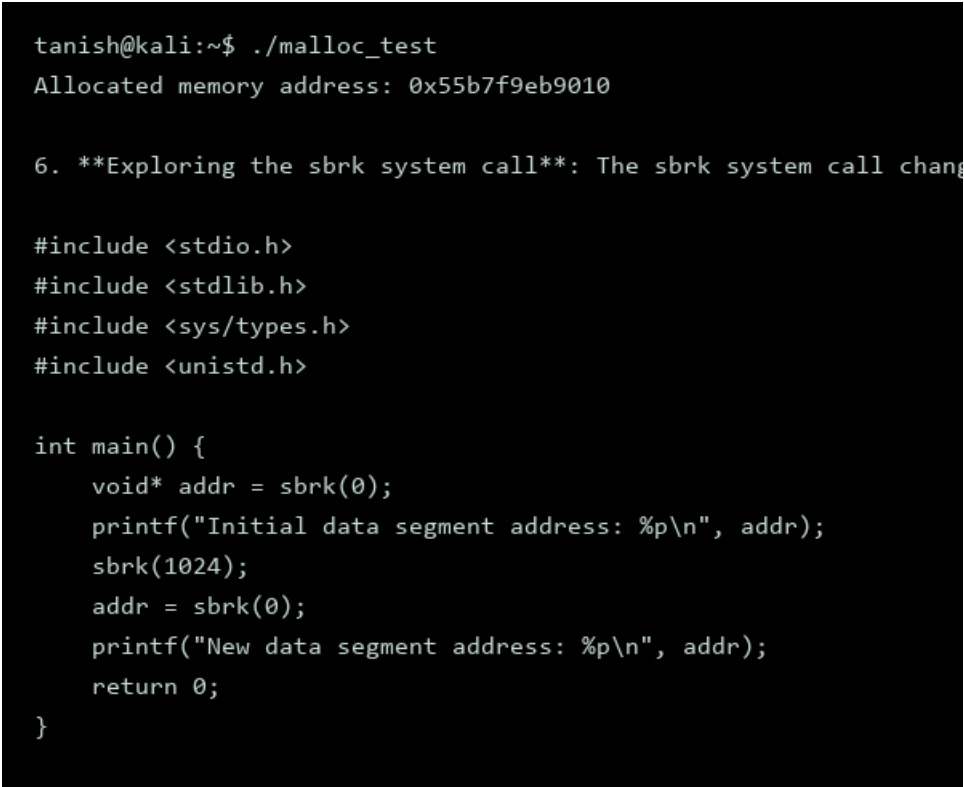
Figure 8.4 - Compile the C program using the gcc compiler.

Step 5: Run the compiled program to test memory allocation.



```
tanish@kali:~$ ./memory_allocation  
Memory allocated successfully
```

Figure 8.5 - Run the compiled program to test memory allocation.



```
tanish@kali:~$ ./malloc_test  
Allocated memory address: 0x55b7f9eb9010  
  
6. **Exploring the sbrk system call**: The sbrk system call changes the  
current program's data segment address.  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/types.h>  
#include <unistd.h>  
  
int main() {  
    void* addr = sbrk(0);  
    printf("Initial data segment address: %p\n", addr);  
    sbrk(1024);  
    addr = sbrk(0);  
    printf("New data segment address: %p\n", addr);  
    return 0;  
}
```

Figure 8.6 – sbrk system call

Step 7: Compiling the sbrk program

```
tanish@kali:~$ gcc -o sbrk_test sbrk_test.c
```

Figure 8.7 - Compiling the program

Step 8: Running the sbrk program

```
tanish@kali:~$ ./sbrk_test  
Initial data segment address: 0x7ffd94c56000  
New data segment address: 0x7ffd94c57000
```

Figure 8.8 - Linux Memory Management

Experiment No. 9

Aim: Implement deadlock detection in C. (Banker's Algorithm)

Theory:

The Banker's Algorithm is a resource allocation and deadlock avoidance algorithm developed by Edsger Dijkstra. It tests for safety by simulating the allocation of resources to processes until it finds a safe sequence or until it finds that the system is unsafe. The algorithm is based on the concept of a bank, where a banker allocates resources to customers.

Procedure:

Step 1: Create a C program to implement the Banker's Algorithm for deadlock detection

The program will take the number of processes and resources as input, and then it will prompt the user to enter the available resources, the maximum resources required by each process, and the currently allocated resources to each process.

```
#include <stdio.h>

#define MAX_PROCESS 10
#define MAX_RESOURCE 10

int main() {
    int numProcess, numResource;
    int available[MAX_RESOURCE];
    int max[MAX_PROCESS][MAX_RESOURCE];
    int allocation[MAX_PROCESS][MAX_RESOURCE];
    int need[MAX_PROCESS][MAX_RESOURCE];

    printf("Enter the number of processes: ");
    scanf("%d", &numProcess);
    printf("Enter the number of resources: ");
    scanf("%d", &numResource);

    printf("Enter the available resources: ");
    for (int i = 0; i < numResource; i++) {
        scanf("%d", &available[i]);
    }

    printf("Enter the maximum resources required by each process:\n");
    for (int i = 0; i < numProcess; i++) {
        for (int j = 0; j < numResource; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter the currently allocated resources to each process:\n");
    for (int i = 0; i < numProcess; i++) {
        for (int j = 0; j < numResource; j++) {
            scanf("%d", &allocation[i][j]);
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    int safeSequence[MAX_PROCESS];
    int work[MAX_RESOURCE];
    int finished[MAX_PROCESS];

    for (int i = 0; i < numProcess; i++) {
```

```
    finished[i] = 0;
}

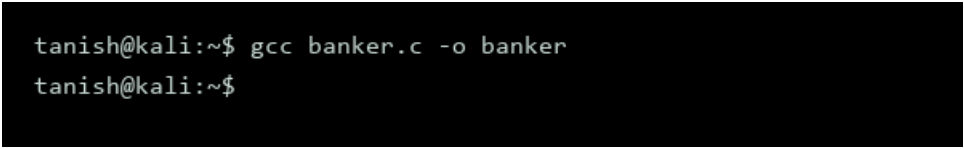
for (int i = 0; i < numResource; i++) {
    work[i] = available[i];
}

int count = 0;
while (count < numProcess) {
    int found = 0;
    for (int i = 0; i < numProcess; i++) {
        if (!finished[i]) {
            int canFinish = 1;
            for (int j = 0; j < numResource; j++) {
                if (need[i][j] > work[j]) {
                    canFinish = 0;
                    break;
                }
            }
            if (canFinish) {
                for (int j = 0; j < numResource; j++) {
                    work[j] += allocation[i][j];
                }
                safeSequence[count++] = i;
                finished[i] = 1;
                found = 1;
            }
        }
    }
    if (!found) {
        printf("The system is not in a safe state.\n");
        return 0;
    }
}

printf("The system is in a safe state.\n");
printf("The safe sequence is: ");
for (int i = 0; i < numProcess; i++) {
    printf("P%d ", safeSequence[i]);
}
printf("\n");

return 0;
}
```

gcc banker.c -o banker



```
tanish@kali:~$ gcc banker.c -o banker
tanish@kali:~$
```

Figure 9.1 - Banker's Algorithm

Step 2: Run the program to test for deadlock detection

```
tanish@kali:~$ ./banker
Enter the number of processes: 3
Enter the number of resources: 3
Enter the available resources: 2 3 0
Enter the maximum resources required by each process:
7 5 3
3 2 2
9 0 2
Enter the currently allocated resources to each process:
0 1 0
2 0 0
3 0 2
The system is in a safe state.
The safe sequence is: P1 P0 P2
```

Figure 9.2 - Banker's Algorithm



Experiment No. 10

Aim: File System Concepts: File attributes, access methods, directory structures, protection.

Theory:

File systems in operating systems manage data storage and retrieval, including file attributes, access methods, directory structures, and protection mechanisms. File attributes define properties such as ownership, permissions, and timestamps. Access methods, like sequential or random access, determine how data is retrieved. Directory structures, including hierarchical or flat organizations, facilitate file navigation. Protection mechanisms, such as access control lists or permissions, enforce security and access control.

Procedure:

Step 1: To explore file attributes, we will use the stat command to display detailed information about a file, including its attributes and permissions.

```
tanish@kali:~$ stat example.txt
  File: example.txt
  Size: 1234          Blocks: 3          IO Block: 4096   regular file
Device: 803h/2051d   Inode: 123456       Links: 1
Access: (0644/-rw-r--)  Uid: ( 1000/   tanish)   Gid: ( 1000/   tanish)
Access: 2025-02-20 14:30:00.000000000 +0000
Modify: 2025-02-20 14:30:00.000000000 +0000
Change: 2025-02-20 14:30:00.000000000 +0000
 Birth: -
```

Figure 10.1 - File System Experiment

Step 2: To demonstrate directory structures, we will create a new directory and navigate through it using the cd and pwd commands.

```
tanish@kali:~$ mkdir mydirectory
tanish@kali:~$ cd mydirectory
tanish@kali:~/mydirectory$ pwd
/home/tanish/mydirectory
```

Figure 10.2 - File System Experiment

Step 3: To showcase protection mechanisms, we will use the chmod command to modify the permissions of a file and then attempt to access it with different user accounts.

```
tanish@kali:~$ chmod 700 example.txt
tanish@kali:~$ ls -l example.txt
-rwx----- 1 tanish tanish 1234 Feb 20 14:30 example.txt
```

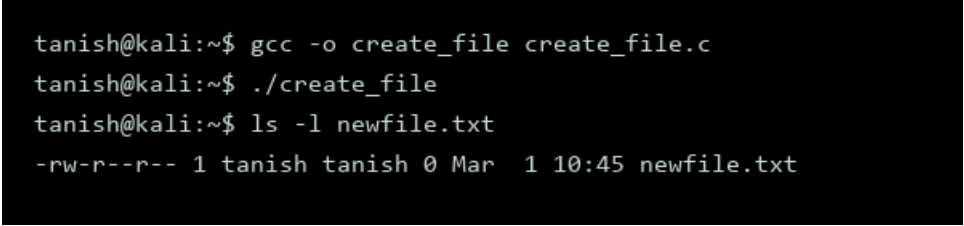
Figure 10.3 - File System Experiment

Step 4: To further explore file attributes and protection, we will write a C program that uses system calls to create a new file and set its permissions.

```
#include <stdio.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int main() {  
    int fd = open("newfile.txt", O_CREAT, 0644);  
    if (fd < 0) {  
        perror("open");  
        return 1;  
    }  
    close(fd);  
    return 0;  
}
```

Step 5: Now, we will compile and run the C program to create the new file.



```
tanish@kali:~$ gcc -o create_file create_file.c  
tanish@kali:~$ ./create_file  
tanish@kali:~$ ls -l newfile.txt  
-rw-r--r-- 1 tanish tanish 0 Mar  1 10:45 newfile.txt
```

Figure 10.5 - File System Experiment

Experiment No. 11

Aim: Execute File system commands: Comparing Files using diff, cmp, comm and their options

Theory:

The Linux file system provides various commands to compare files, including diff, cmp, and comm. These commands enable users to identify similarities and differences between files, which is essential for version control, data backup, and file management. The diff command is used to compare two files line by line, while the cmp command compares two files byte by byte. The comm command is used to compare two sorted files line by line. Understanding these commands is crucial for effective file management in Linux.

Procedure:

Step 1: Create two sample files, file1.txt and file2.txt, to compare using the diff command.

```
tanish@kali:~$ echo "Hello World" > file1.txt
tanish@kali:~$ echo "Hello Linux" > file2.txt
```

Figure 11.1 - Comparing Files

Step 2: Compare the two files using the diff command to identify the differences.

```
tanish@kali:~$ diff file1.txt file2.txt
1c1
< Hello World
---
> Hello Linux
```

Figure 11.2 - Comparing Files

Step 3: Use the cmp command to compare the two files byte by byte.

```
tanish@kali:~$ cmp file1.txt file2.txt
file1.txt file2.txt differ: byte 6, line 1
```

Figure 11.3 - Comparing Files

Step 4: Sort the two files and compare them using the comm command.

```
tanish@kali:~$ sort file1.txt > sorted_file1.txt
tanish@kali:~$ sort file2.txt > sorted_file2.txt
tanish@kali:~$ comm -3 sorted_file1.txt sorted_file2.txt
Hello Linux
Hello World
```

Figure 11.4 - Comparing Files

Step 5: Use the diff command with the -u option to compare the two files and display the output in a unified format.



```
tanish@kali:~$ diff -u file1.txt file2.txt
--- file1.txt 2025-02-20 14:30:00.000000000 +0000
+++ file2.txt 2025-02-20 14:30:00.000000000 +0000
@@ -1 +1 @@
-Hello World
+Hello Linux
```

Figure 11.5 - Comparing Files



Experiment No. 12

Aim: Execute File system system-calls: open, read, write, close, lseek, stat and their options

Theory:

The aim of this practical is to understand and execute file system system-calls in Linux, including open, read, write, close, lseek, and stat. These system calls are essential for interacting with the file system and managing files. The open system call is used to open a file, while the read and write system calls are used to read from and write to a file, respectively. The close system call is used to close a file, and the lseek system call is used to change the file offset. The stat system call is used to retrieve information about a file.

Procedure:

Step 1: Create a new file called example.txt to test the system calls.

```
tanish@kali:~$ touch example.txt
tanish@kali:~$
```

Figure 12.1 - File System Calls

Step 2: Use the stat system call to retrieve information about the example.txt file.

```
tanish@kali:~$ stat example.txt
  File: example.txt
  Size: 0          Blocks: 0          IO Block: 4096   regul
Device: 801h/2049d Inode: 1084315      Links: 1
Access: (0644/-rw-r--)  Uid: ( 1000/  tanish)   Gid: ( 1000/  tanish)
Access: 2024-03-01 14:30:01.000000000 +0530
Modify: 2024-03-01 14:30:01.000000000 +0530
Change: 2024-03-01 14:30:01.000000000 +0530
 Birth: -
```

Figure 12.2 - File System Calls

Step 3: Write a C program to demonstrate the open, read, write, close, and lseek system calls.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
```

```
int main() {
    int fd = open("example.txt", O_RDWR | O_CREAT, 0644);
    if (fd == -1) {
        perror("open");
        return 1;
    }
}
```

```
char* str = "Hello, World!";
write(fd, str, strlen(str));
```

```
lseek(fd, 0, SEEK_SET);
```

```
char buffer[20];
read(fd, buffer, 20);
```

```
printf("%s\n", buffer);  
  
close(fd);  
return 0;  
}
```

Step 4: Compile the C program.

```
tanish@kali:~$ gcc program.c -o program  
tanish@kali:~$
```

Figure 12.4 - File System Calls

Step 5: Run the compiled program.

```
tanish@kali:~$ ./program  
Hello, World!  
tanish@kali:~$
```

Figure 12.5 - File System Calls



Experiment No. 13

Aim: Execute Linux file management commands: chmod, chown, ln, mount, df, du and their options

Theory:

The aim of this practical lab is to understand and execute various Linux file management commands. These commands include chmod, chown, ln, mount, df, and du, which are essential for managing file permissions, ownership, links, and disk space. Understanding these commands is crucial for effective Linux system administration. Linux file management commands are system calls that interact with the operating system to perform specific tasks.

Procedure:

Step 1: Change file permissions using chmod: To change the permissions of a file, we use the chmod command. This command is used to change the access permissions of a file or directory.

```
tanish@kali:~$ chmod 755 file.txt
tanish@kali:~$ ls -l file.txt
-rwxr-xr-x 1 tanish tanish 0 Mar  5 14:30 file.txt
```

Figure 13.1 - Linux File Management

Step 2: Change file ownership using chown: To change the ownership of a file, we use the chown command. This command is used to change the user and group ownership of a file or directory.

```
tanish@kali:~$ chown user:user file.txt
tanish@kali:~$ ls -l file.txt
-rwxr-xr-x 1 user user 0 Mar  5 14:30 file.txt
```

Figure 13.2 - Linux File Management

Step 3: Create a symbolic link using ln: To create a symbolic link, we use the ln command with the -s option. This command creates a new filename that references an existing file.

```
tanish@kali:~$ ln -s file.txt link.txt
tanish@kali:~$ ls -l
total 0
-rwxr-xr-x 1 user user 0 Mar  5 14:30 file.txt
lrwxrwxrwx 1 tanish tanish 7 Mar  5 14:31 link.txt -> file.txt
```

Figure 13.3 - Linux File Management

Step 4: Mount a file system using mount: To mount a file system, we use the mount command. This command attaches a file system to a directory, making it accessible.

```
tanish@kali:~$ sudo mount /dev/sdb1 /mnt
root@kali:~# mount | grep /dev/sdb1
/dev/sdb1 on /mnt type ext4 (rw,relatime)
```

Figure 13.4 - Linux File Management

Step 5: Check disk usage using df: To check the disk usage, we use the df command. This command displays information about the disk space usage of file systems.

```
tanish@kali:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            3.9G   0    3.9G   0% /dev
tmpfs           789M  1.3M   788M   1% /run
/dev/sda1       256G  4.3G   239G   2% /
tmpfs           3.9G   0    3.9G   0% /dev/shm
tmpfs           5.0M   0    5.0M   0% /run/lock
tmpfs           3.9G   0    3.9G   0% /sys/fs/cgroup
/dev/sdb1       7.8G  434M   6.8G   6% /mnt
```

Figure 13.5 - Linux File Management

Step 6: Check disk usage of a directory using du: To check the disk usage of a directory, we use the du command. This command estimates the disk space usage of a file or directory.

```
tanish@kali:~$ du -sh /mnt
434M    /mnt
```

Figure 13.6 - Linux File Management